

Centro educativo

Código	Centro	Concello	Ano académico
15005397	I.E.S. Fernando Wirtz Suárez	A Coruña	2025-2026

Ciclo formativo

Código da familia profesional	Familia profesional	Código do ciclo formativo	Ciclo formativo	Grao	Réxime
FP16	Informática e comunicacións	CSIFC01	Desenvolvemento de Aplicacións Web	Superior	Ordinario

Módulo profesional e unidades formativas de menor duración (*)

Código MP/UF	Nome
MP0616	Proxecto intermodular de desenvolvemento de aplicacións web

Profesorado responsable

Titor	Rodrigo Dopazo Iglesias
-------	-------------------------

Alumno

Alumno	Mihai Taran
--------	-------------

Datos do Proxecto

Título	RoomMate: Desarrollo de una aplicación web para la gestión integral de pisos compartidos
--------	--

CONTROL DE VERSIONS:

Versión	Data	Observacións
1.0	30/05/2026	Versión inicial del documento

ABSTRACT

RoomMate: A Full-Stack Web Application for Shared Housing Management

Shared housing has become highly common among students and young professionals, centralised tool exists to manage the day aspects of cohabitation. This project addresses that gap by developing RoomMate, a full-stack web application designed to optimize the management of shared flats between landlords and tenants. The application aims to provide a unified platform where landlords can manage properties, assign tenants, register and split expenses, track payments, handle incidents, and organise household tasks, while tenants can view their pending payments, report issues, participate in assigned tasks, and communicate with flatmates through a real-time chat. The system was built using ASP.NET Core (.NET 10) for the REST API backend, Blazor WebAssembly for the frontend, and SQL Server as the database managed through Entity Framework Core with a Code First approach. Authentication is handled via JWT tokens, real-time messaging through SignalR, and address autocomplete and nearby places search through OpenStreetMap's Nominatim and Overpass APIs. The application was fully containerised using Docker and deployed on a Hetzner VPS accessible at roommate-app.duckdns.org. All primary objectives were successfully met, resulting in a functional, responsive, and deployable web application that effectively covers the core needs of shared housing management.

Keywords: Blazor WebAssembly, ASP.NET Core, shared housing management, JWT authentication, Docker deployment.

Índice:

1.	Objetivo	4
2.	Descripción	4
3.	Alcance	5
4.	Planificación	6
5.	Medios a emplegar	7
6.	Presupuesto	9
7.	Título	10
8.	Execución	11

1. Obxectivo

El objetivo de este proyecto es desarrollar una aplicación web completa para la gestión de pisos compartidos, denominada RoomMate, que permita a propietarios e inquilinos gestionar de forma centralizada y eficiente todos los aspectos relacionados con la convivencia en una vivienda compartida.

La aplicación debe permitir a los propietarios gestionar sus viviendas, asignar inquilinos, registrar y repartir gastos, controlar el estado de los pagos, gestionar incidencias y organizar tareas del hogar. Los inquilinos, por su parte, deben poder consultar sus pagos pendientes, reportar incidencias, participar en las tareas asignadas y comunicarse con el resto de habitantes a través de un chat en tiempo real.

El éxito del proyecto se determina por el cumplimiento de los siguientes criterios:

- Implementación de una API REST con ASP.NET Core que gestione todas las entidades del sistema con autenticación JWT.
- Desarrollo de un frontend funcional con Blazor WebAssembly que se adapte a dispositivos móviles y de escritorio.
- Sistema de autenticación con roles diferenciados para propietario e inquilino.
- Chat en tiempo real mediante SignalR.
- Integración con APIs externas de OpenStreetMap para geolocalización y lugares cercanos.
- Despliegue completo de la aplicación mediante Docker con base de datos incluida.
- Publicación de la aplicación en un servidor de producción accesible desde internet.

2. Descripción

RoomMate se crea en base a la necesidad real de simplificar la gestión de pisos compartidos, una situación cada vez más común hoy en día entre estudiantes y trabajadores. La motivación personal detrás del proyecto surge de haber vivido en primera persona la dificultad de organizar gastos, tareas y comunicación entre compañeros de piso sin una herramienta centralizada que lo facilite. Por la falta de algunas herramientas de gestión que tiene esta aplicación web se puede llegar hasta a malentendidos, pues a veces la comunicación entre propietarios e inquilinos no se lleva de la mejor forma posible, ni tampoco se registra.

La solución adoptada es una aplicación web full-stack compuesta por una API REST desarrollada con ASP.NET Core (.NET 10) y un frontend con Blazor Web Assembly, comunicados mediante HTTP con autenticación JWT. La base de datos es SQL Server gestionada con Entity Framework Core en modo Code First, y la aplicación completa está dockerizada para facilitar su despliegue en cualquier entorno.

La arquitectura se divide en cuatro capas principales bien diferenciadas: API (controllers y hubs), Servicios (lógica de negocio), Repositorios (acceso a datos) y Cliente (Blazor WebAssembly), lo que permite una separación clara de responsabilidades y facilita el mantenimiento y la escalabilidad del proyecto.

Entre los aspectos más relevantes y que mayor dificultad supusieron durante el desarrollo destacan los siguientes, que se desarrollarán en detalle en apartados posteriores:

La autenticación con JWT fue uno de los puntos más difíciles de conseguir, ya que requirió implementar la generación y validación de tokens en el backend, gestionar su almacenamiento seguro en el cliente mediante Blazored.LocalStorage, crear un AuthHandler que inyecta automáticamente el token en cada petición HTTP, y controlar la expiración del token para redirigir al usuario al login cuando fuera necesario.

La integración con las APIs de OpenStreetMap fue otro gran reto, especialmente la geocodificación inversa mediante Nominatim para obtener coordenadas a partir de direcciones, y el sistema de autocompletado de direcciones en el formulario de viviendas, que debía mostrar sugerencias en tiempo real mientras el usuario escribe rellenando automáticamente ciudad y código postal al seleccionar una opción.

El calendario de tareas fue otro de los puntos más elaborados, ya que debía representar visualmente un calendario mensual con navegación entre meses, mostrar las tareas programadas en sus días correspondientes, gestionar tareas recurrentes calculando en qué días debían aparecer según su frecuencia en días, y permitir crear, completar y eliminar tareas con diferentes permisos según el rol del usuario.

El resto de funcionalidades, aunque igualmente relevantes, resultaron más directas en su implementación gracias a la arquitectura definida desde el inicio del proyecto.

3. Alcance

Durante el desarrollo se identificaron funcionalidades adicionales que mejorarían la experiencia de usuario y que se contemplan como mejoras futuras para versiones posteriores de la aplicación:

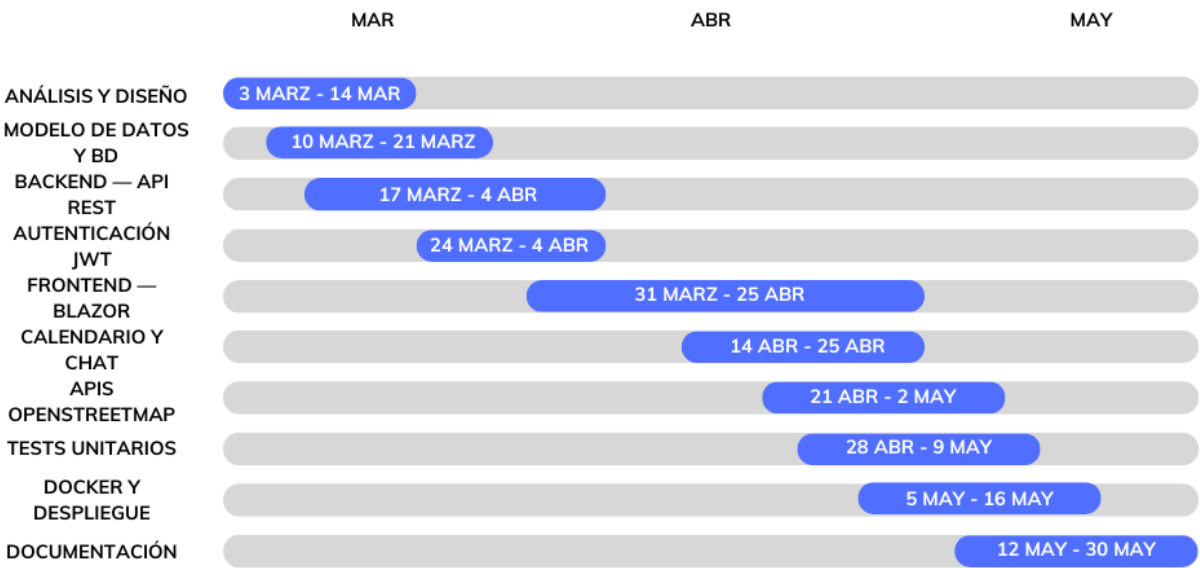
Sistema de notificaciones por email: Se contempló la posibilidad de enviar correos electrónicos automáticos en eventos clave como la validación de cuenta al registrarse, la asignación de un inquilino a una vivienda, la creación de un pago pendiente, la asignación de una tarea o el reporte de una incidencia. Esta funcionalidad requeriría integrar un servicio de envío de emails como SendGrid o SMTP y mejoraría significativamente la experiencia de usuario al mantenerlo informado sin necesidad de acceder a la aplicación. Toda la información está disponible dentro de la propia aplicación, por lo que su ausencia no condiciona el funcionamiento del sistema.

Gestión de archivos multimedia: Se planteó permitir subir imágenes y documentos PDF en diferentes contextos: fotos de perfil para personalizar la cuenta del usuario, imágenes o facturas en PDF adjuntas a los gastos como justificante, y capturas de transferencias o pagos por Bizum adjuntas a los pagos como prueba del mismo. Esta funcionalidad requeriría implementar un sistema de almacenamiento de archivos en el servidor. Su ausencia no impide el uso de la aplicación pero limitaría la trazabilidad documental de los gastos y pagos entre los usuarios.

Ambas funcionalidades quedan definidas como líneas de trabajo futuro que ampliarían las capacidades de RoomMate sin afectar al núcleo funcional ya implementado.

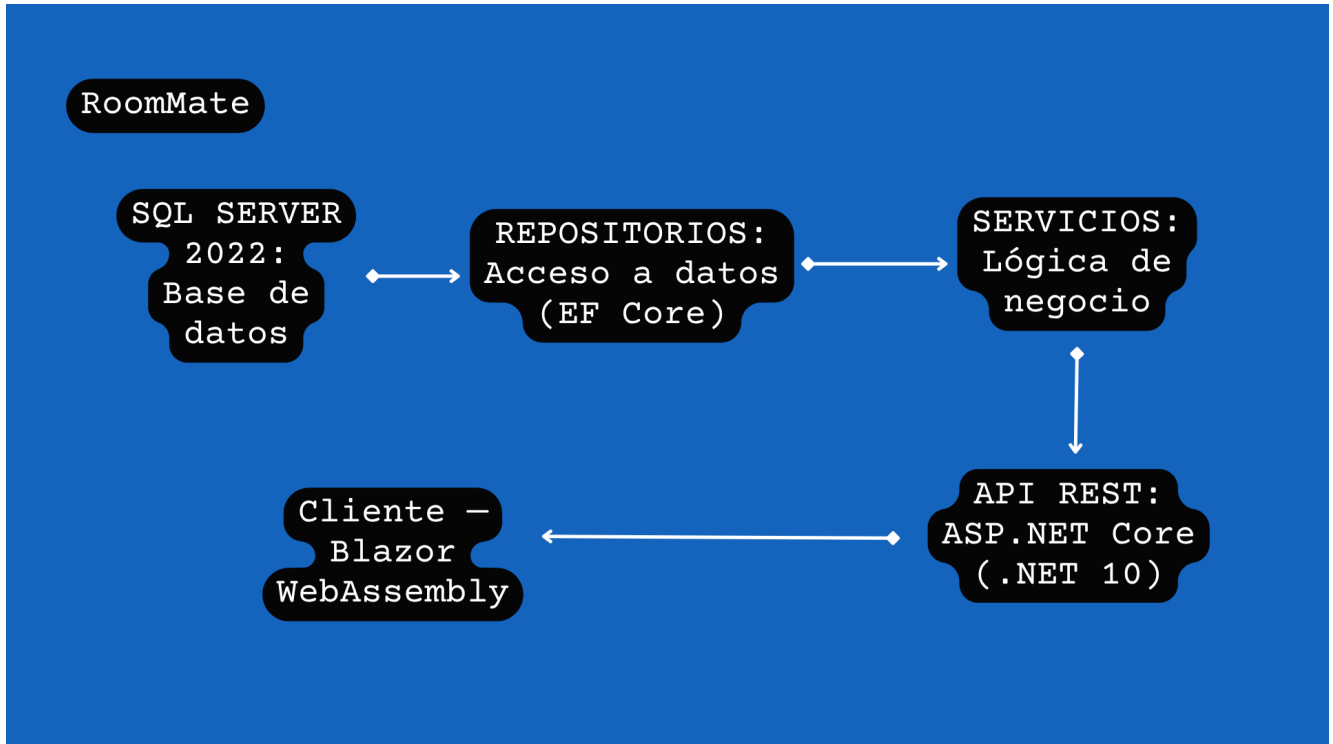
4. Planificación

GRÁFICO GANTT



La planificación del proyecto se organizó en 12 semanas distribuidas desde principios de marzo hasta finales de mayo. Se comenzó por el análisis de requisitos y el diseño del modelo de datos, para después construir el backend y el frontend de forma progresiva. Las fases de mayor dedicación en cuanto a complejidad fueron las de la autenticación JWT, el diseño del frontend Blazor y la integración con las APIs externas. La fase de Docker y despliegue se dejó para el final una vez la aplicación estaba completamente funcional, y la documentación se elaboró en paralelo durante las últimas semanas.

5. Medios a emplegar



Hardware

El desarrollo se realizó en un ordenador personal con Windows 11. Para el despliegue en producción se contrató un servidor VPS de Hetzner (modelo CX23, 2 vCPU, 4GB RAM) ubicado en Helsinki, accesible mediante SSH desde el ordenador de desarrollo.

Stack tecnológico y justificación

Se eligió ASP.NET Core (.NET 10) para el backend por ser un framework moderno, de alto rendimiento y multiplataforma, con soporte nativo para APIs REST, autenticación JWT y SignalR. Al estar desarrollado en C#, permite compartir los modelos y DTOs entre el backend y el frontend sin duplicar código.

Blazor WebAssembly se eligió como framework de frontend por permitir desarrollar toda la aplicación en C# sin necesidad de JavaScript, lo que supone una ventaja al trabajar con un único lenguaje en todo el proyecto. Al ejecutarse en el navegador mediante WebAssembly, ofrece una experiencia de aplicación de página única (SPA) sin depender de un servidor para renderizar las vistas.

MudBlazor se usó como biblioteca de componentes UI por su amplia colección de componentes listos para usar (tablas, formularios, diálogos, calendario, etc.) compatibles con Blazor WebAssembly, lo que permitió construir una interfaz responsive y visualmente cuidada sin necesidad de escribir CSS complejo desde cero.

Entity Framework Core se eligió como ORM para el acceso a datos por su integración nativa con .NET, su soporte para Code First con migraciones automáticas y su capacidad para abstraer las consultas SQL mediante LINQ. Esto permite trabajar directamente con objetos C# en vez de escribir SQL manualmente.

SQL Server se usó como base de datos por su compatibilidad total con Entity Framework Core y por ser el sistema de bases de datos más extendido en el ecosistema .NET. Se gestionó localmente durante el desarrollo con SQL Server Management Studio (SSMS), que proporciona una interfaz gráfica para inspeccionar tablas, ejecutar consultas y verificar las migraciones aplicadas.

SignalR (biblioteca de código abierto que facilita la incorporación de la funcionalidad de comunicación en tiempo real en sus aplicaciones) se integró para implementar el chat en tiempo real entre los usuarios de una misma vivienda. Permite mantener una conexión persistente entre el cliente y el servidor mediante WebSockets, de forma que los mensajes se reciben instantáneamente sin necesidad de recargar la página.

Las APIs de OpenStreetMap (Nominatim y Overpass) se eligieron por ser gratuitas y no requerir API key, lo que simplifica la integración. Nominatim se usa para geocodificación y autocompletado de direcciones, y Overpass para buscar lugares de interés cercanos a una vivienda.

JWT (JSON Web Tokens) se usó para la autenticación sin estado (stateless), lo que permite que el servidor no necesite mantener sesiones. El token se genera en el login, se almacena en el cliente con Blazored.LocalStorage y se envía automáticamente en cada petición mediante un AuthHandler personalizado.

Herramientas de desarrollo

Visual Studio 2026 fue el entorno de desarrollo principal por su integración con .NET, su depurador, y su soporte para proyectos Blazor y ASP.NET Core.

GitHub se usó como sistema de control de versiones, trabajando con dos ramas principales: develop para el desarrollo activo y master para las versiones estables. Esto permitió mantener un historial de cambios claro y revertir modificaciones cuando fue necesario.

Postman se usó para probar y depurar los endpoints de la API durante el desarrollo, permitiendo enviar peticiones HTTP con cabeceras de autenticación y verificar las respuestas antes de integrarlos en el frontend.

Docker se eligió para contenerizar la aplicación completa (API, Client y SQL Server) en tres contenedores orquestados con docker-compose. Esto permite que cualquier persona pueda levantar la aplicación en su máquina con un único comando sin necesidad de instalar .NET, SQL Server ni ninguna dependencia adicional. También facilita el despliegue en producción.

Despliegue

Para el despliegue en producción se eligió Hetzner como proveedor de VPS por su buena relación calidad-precio y sus servidores ubicados en Europa. La aplicación se desplegó clonando el repositorio de GitHub en el servidor, ejecutando docker compose up --build y configurando un subdominio gratuito con DuckDNS (roommate-app.duckdns.org) que apunta a la IP del servidor, haciendo la aplicación accesible desde internet sin coste adicional.

6. Presupuesto

Recursos humanos

El proyecto fue desarrollado por una sola persona durante aproximadamente 12 semanas. Considerando un perfil de desarrollador web junior con una tarifa estimada de 15€/hora:

Concepto	Horas	Tarifa	Total
Análisis y diseño	15h	15€/h	225€
Desarrollo backend	45h	15€/h	675€
Desarrollo frontend	50h	15€/h	750€
Testing y pruebas	10h	15€/h	150€
Docker y despliegue	15h	15€/h	225€
Documentación	15h	15€/h	225€
Total horas	150h		2.250€

Recursos materiales y software

Concepto	Coste
Ordenador personal (amortización)	0€
Visual Studio 2026 (Community, gratuito)	0€
SQL Server Management Studio (gratuito)	0€
GitHub (plan gratuito)	0€
APIs OpenStreetMap (gratuitas)	0€
DuckDNS (gratuito)	0€
Servidor Hetzner CX23 (estimado 2 meses)	8€
Total materiales	8€

Presupuesto total estimado

Concepto	Total
Recursos humanos	2.250€
Recursos materiales	8€
Total	2.258€

Cabe destacar que el coste real del proyecto en cuanto a licencias y herramientas es prácticamente nulo, ya que todas las tecnologías utilizadas son gratuitas o de código abierto. El único coste operativo es el del servidor de producción, que además es opcional para el funcionamiento local de la aplicación mediante Docker.

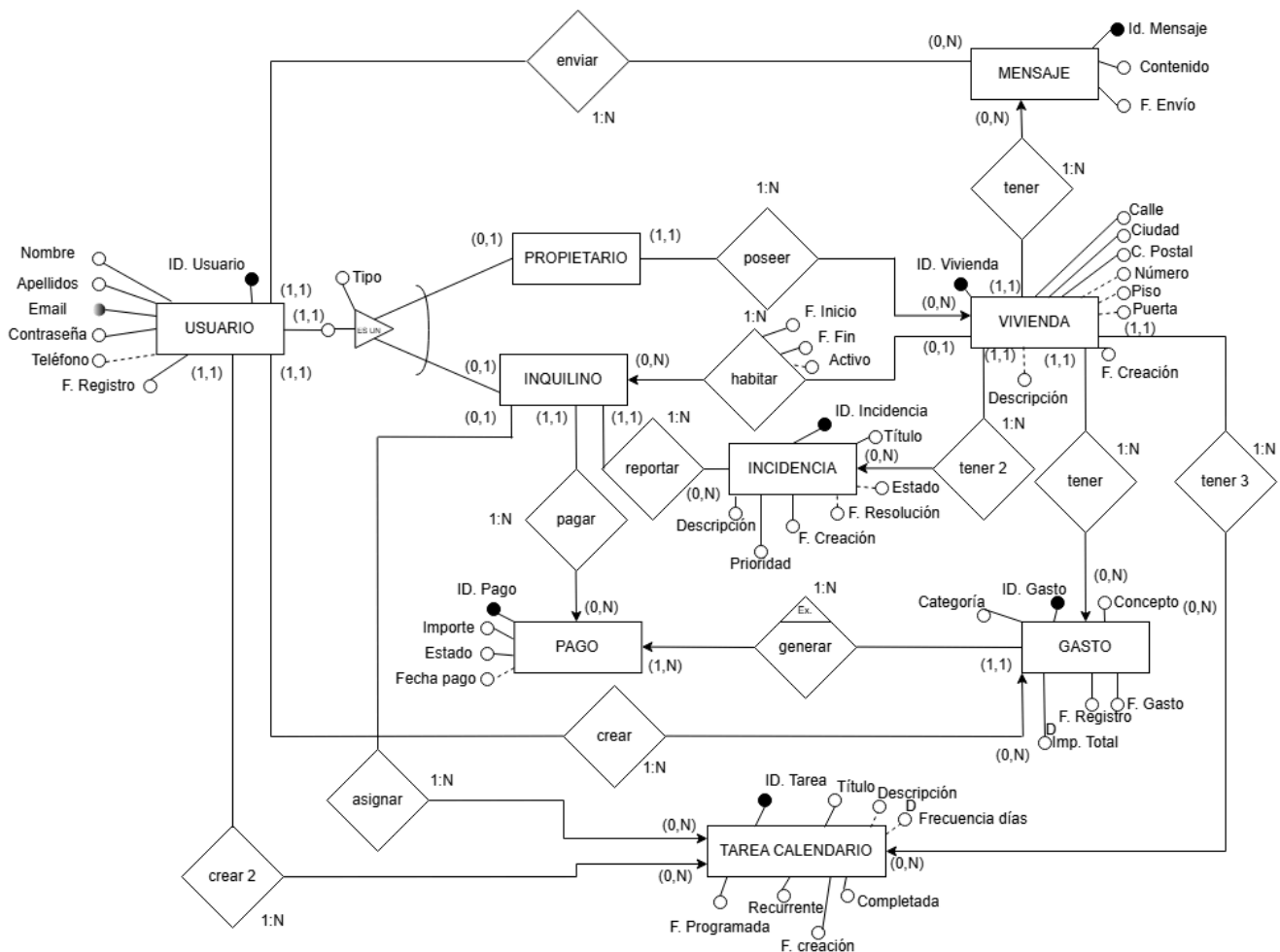
7. Título

El título del proyecto es:

RoomMate: Desarrollo de una aplicación web para la gestión integral de pisos compartidos

8. Ejecución

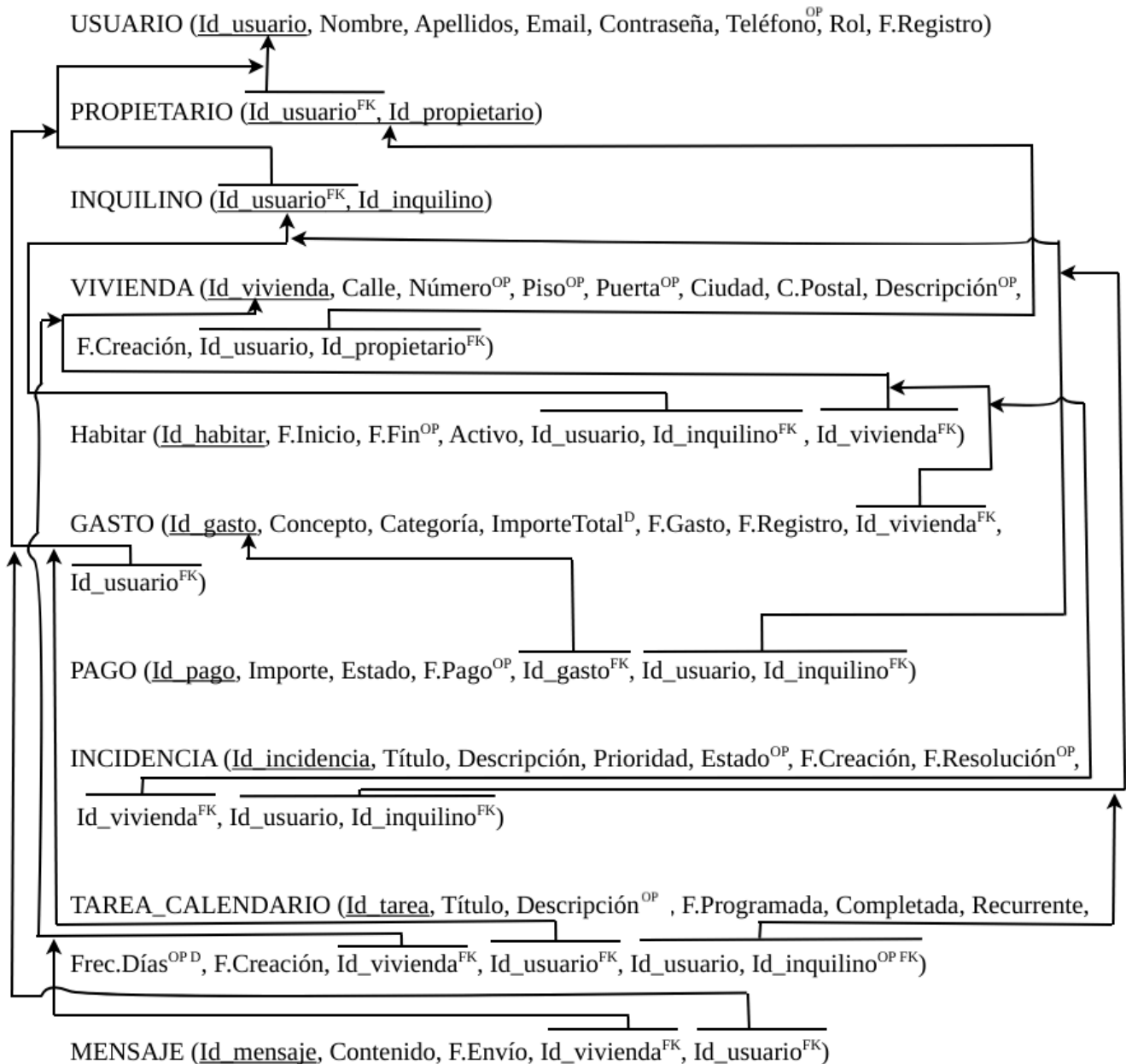
Diagrama ER + DD:



1. Los atributos ID de todas las entidades fueron escogidos como claves principales ya que son únicos para cada entidad y se generan automáticamente mediante autoincremento en la base de datos.
2. El atributo Email de USUARIO es clave candidata, pues es único para cada usuario y se valida en el registro para evitar duplicados.
3. Los atributos Descripción en VIVIENDA, INCIDENCIA y TAREA_CALENDARIO guardan información adicional opcional sobre la entidad a la que hacen referencia.
4. PAGO se considera entidad débil por existencia respecto a GASTO, ya que un pago no puede existir sin su gasto asociado. Si se elimina un gasto, sus pagos se eliminan en cascada.
5. El atributo ImporteTotal de GASTO es derivado, ya que puede calcularse sumando los importes de todos sus pagos asociados.

6. El atributo Activo de la interrelación habitar indica si la asignación del inquilino a la vivienda está vigente. Tiene como dominio: true si el inquilino sigue residiendo en la vivienda y false si ya no reside. Esto permite mantener el historial de residencias.
7. El atributo Estado de PAGO tiene como dominio: Pendiente cuando el pago no ha sido abonado, Pagado cuando el inquilino lo ha marcado como pagado y Rechazado si el propietario lo rechaza.
8. El atributo Estado de INCIDENCIA refleja el progreso de resolución. Tiene como dominio: Abierta, EnProceso, Resuelta y Cerrada. Solo el propietario puede cambiar el estado.
9. El atributo Prioridad de INCIDENCIA indica la urgencia. Tiene como dominio: Baja, Media, Alta y Urgente. Lo establece el inquilino al reportar la incidencia.
10. El atributo Recurrente de TAREA_CALENDARIO indica si la tarea se repite periódicamente.
11. El atributo FrecuenciaDias es opcional y derivado de Recurrente, sólo tiene valor si Recurrente es true, indicando cada cuántos días se repite la tarea.
12. El atributo Tipo de USUARIO implementa la especialización total exclusiva entre PROPIETARIO e INQUILINO. Tiene como dominio: Propietario o Inquilino. Un usuario no puede tener ambos roles simultáneamente.
13. El atributo FechaFin de la interrelación habitar es opcional, ya que solo se rellena cuando el inquilino deja de residir en la vivienda.
14. El atributo Fecha pago de Pago también es opcional, pues se completa al completar el pago.
15. El atributo Completada de TAREA_CALENDARIO tiene como dominio: false cuando la tarea está pendiente y true cuando ha sido completada por algún usuario con permisos.

Relacional + DD:

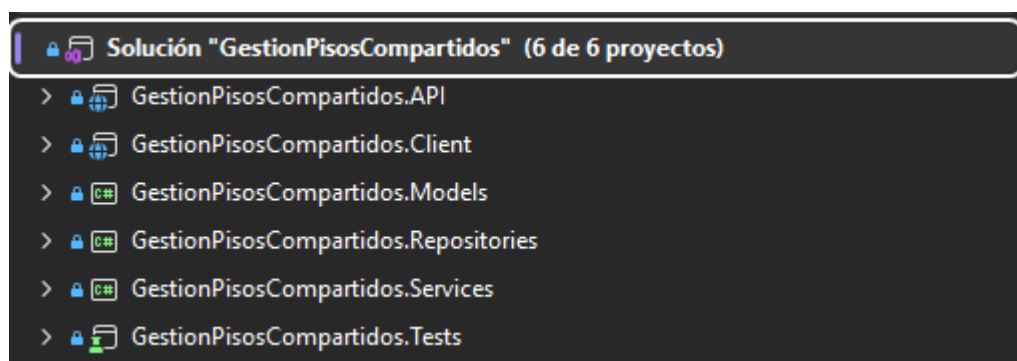


1. Se comenzó por la entidad USUARIO, ya que es la entidad central del sistema y de la que dependen todas las demás. Una vez creada se pudieron generar las entidades especializadas PROPIETARIO e INQUILINO, y a partir de ellas el resto de entidades con sus relaciones.

2. La especialización USUARIO → PROPIETARIO / INQUILINO es total exclusiva, ya que todo usuario debe ser obligatoriamente de uno de los dos tipos y no puede serlo de ambos simultáneamente. En el modelo relacional se representa con dos tablas adicionales PROPIETARIO e INQUILINO que referencian a USUARIO mediante clave foránea. Sin embargo, en la implementación real con Entity Framework Core se optó por una única tabla Usuarios con el atributo Rol que determina el tipo de usuario, ya que EF Core gestiona mejor la herencia mediante una sola tabla.

3. La interrelación HABITAR entre INQUILINO y VIVIENDA generó su propia tabla llamada Habitar en el modelo relacional. En la base de datos esta tabla se denomina InquilinosViviendas siguiendo la convención de nomenclatura de Entity Framework Core, pero conceptualmente representa la interrelación HABITAR del diagrama relacional.
4. Los atributos opcionales se indican con el superíndice OP. En VIVIENDA son opcionales Número, Piso, Puerta y Descripción. En USUARIO es opcional Teléfono. En PAGO es opcional F.Pago, ya que solo se rellena cuando el inquilino marca el pago como realizado. En INCIDENCIA son opcionales Descripción y F.Resolución.
5. El atributo ImporteTotal de GASTO se marca como derivado ya que teóricamente podría calcularse sumando los importes de todos sus pagos asociados, aunque se guarda directamente en la base de datos para optimizar las consultas.
6. El atributo Frec.Días de TAREA_CALENDARIO es opcional y derivado (superíndice OP D), ya que solo tiene valor cuando el atributo Recurrente es verdadero, indicando cada cuántos días se repite la tarea.
7. TAREA_CALENDARIO tiene dos referencias a USUARIO mediante claves foráneas: Id_usuario que referencia al usuario que creó la tarea, e Id_inquilino que referencia al inquilino al que está asignada, siendo este último opcional ya que una tarea puede existir sin estar asignada a nadie.
8. De forma similar, INCIDENCIA tiene dos referencias: Id_vivienda para la vivienda a la que pertenece la incidencia, e Id_inquilino para el inquilino que la reportó.
9. GASTO tiene dos referencias a otras entidades: Id_vivienda para la vivienda a la que pertenece el gasto, e Id_usuario para el usuario que lo creó, pudiendo ser este tanto propietario como inquilino.
10. PAGO es la entidad más dependiente del modelo, ya que referencia tanto a GASTO (del que depende por existencia) como a INQUILINO (que es quien debe realizar el pago). Si se elimina un GASTO, sus PAGOS asociados se eliminan en cascada.
11. MENSAJE referencia tanto a VIVIENDA como a USUARIO, permitiendo así identificar a qué vivienda pertenece el mensaje y quién lo envió.

Estructura de la solución:



La solución GestionPisosCompartidos está compuesta por 6 proyectos organizados en capas con responsabilidades bien diferenciadas:

GestionPisosCompartidos.API: Proyecto principal de arranque. Contiene los controllers REST, el hub de SignalR para el chat, la configuración de JWT, CORS, EF Core y el Program.cs que registra todos los servicios y levanta la aplicación.

GestionPisosCompartidos.Client: Frontend de la aplicación desarrollada con Blazor WebAssembly. Contiene todas las páginas Razor, los servicios del cliente (AuthService, AuthHandler), los DTOs del cliente, los helpers (FormatHelper, ColorHelper), la configuración del tema MudBlazor y los archivos estáticos.

GestionPisosCompartidos.Models: Librería compartida entre el backend y el cliente. Contiene las entidades de la base de datos y los DTOs usados para la comunicación entre capas.

GestionPisosCompartidos.Repositories: Capa de acceso a datos. Contiene el ApplicationDbContext, las migraciones de Entity Framework Core y las implementaciones del patrón repositorio con sus interfaces.

GestionPisosCompartidos.Services: Capa de lógica de negocio. Contiene los servicios con las reglas de negocio de la aplicación y sus interfaces, incluyendo el LugaresService que integra las APIs externas de OpenStreetMap.

GestionPisosCompartidos.Tests: Proyecto de tests unitarios con xUnit, Moq y FluentAssertions que prueba los servicios más críticos de la aplicación.

A continuación se explicará cada una de las capas del proyecto entrando más en detalle en algunos de sus funcionamientos:

Repositories:

DbContext: Esta es la clase central de Entity Framework Core que representa la sesión con la base de datos. Hereda de DbContext y contiene un DbSet<T> por cada entidad del modelo, que se traduce en una tabla en la base de datos. También es en esta clase donde se configura el comportamiento de las relaciones entre entidades mediante el método OnModelCreating, donde se definen las claves foráneas, las restricciones de borrado en cascada y cualquier configuración adicional del modelo.

Migraciones: Las migraciones son el mecanismo que proporciona Entity Framework Core para mantener el esquema de la base de datos sincronizado con el modelo de clases C#. Funcionan de la siguiente manera: cada vez que se modifica una entidad (añadir un campo, cambiar un tipo, eliminar una columna) se genera una nueva migración que describe exactamente qué cambios hay que aplicar a la base de datos.

Para crear una migración se ejecuta el siguiente comando desde la consola del gestor de paquetes de Visual Studio:

```
PM> Add-Migration NombreDeLaMigracion -Project GestionPisosCompartidos.Repositories -StartupProject GestionPisosCompartidos.API
```

Y para aplicarla a la base de datos:

```
PM> Update-Database -Project GestionPisosCompartidos.Repositories -StartupProject GestionPisosCompartidos.API
```

Services:

Entre las reglas de negocio más destacadas implementadas en los servicios se encuentran: un inquilino solo puede tener una vivienda activa simultáneamente, no se puede eliminar una vivienda con inquilinos activos, un inquilino no puede crear gastos de categoría alquiler, y al eliminar un gasto se eliminan en cascada todos sus pagos asociados.

AuthService: Este servicio es el encargado de gestionar el registro y la autenticación de usuarios. En el registro comprueba que no existe otro usuario con el mismo email y hashea la contraseña usando BCrypt antes de guardarla. En el login verifica la contraseña contra el hash almacenado y si es correcta genera un token JWT mediante JwtSecurityToken, incluyendo claims como el ID del usuario, su email, su nombre y su rol. El token se firma con una clave secreta usando el algoritmo HS256 y tiene una validez configurable mediante appsettings.json. Este token se devuelve al cliente junto con los datos del usuario para que pueda autenticarse en las siguientes peticiones.

LugaresService: Este servicio integra las APIs externas de OpenStreetMap para dos funcionalidades. La primera es el autocompletado de direcciones mediante la API de Nominatim: recibe el texto que el usuario escribe, construye una query añadiendo España como filtro geográfico y devuelve sugerencias con calle, número, ciudad y código postal extraídos del campo address de la respuesta. La segunda es la búsqueda de lugares cercanos mediante la API de Overpass: recibe las coordenadas de la vivienda y el tipo de lugar buscado, construye una query en el lenguaje propio de Overpass buscando en un radio de 1500 metros y parsea la respuesta JSON para devolver los resultados. El servicio incluye un método ConvertirTipo que traduce los tipos de lugar de la aplicación a los tags correspondientes de OpenStreetMap.

API:

En esta capa se encuentran todos los controllers REST necesarios. Estos endpoints reciben las peticiones HTTP del cliente, delegan la lógica en los servicios correspondientes mediante inyección de dependencias y devuelven las respuestas en formato JSON. Todos los endpoints están protegidos con el atributo [Authorize] salvo los de autenticación (login y registro), de forma que solo los usuarios con un token JWT válido pueden acceder a ellos. Además comprueban el rol del usuario para restringir el acceso según si es propietario o inquilino.

Program.cs (Client): Es el punto de entrada del backend. En él se registran todos los servicios necesarios: los repositorios y servicios mediante inyección de dependencias, el ApplicationDbContext con la cadena de conexión a SQL Server, la autenticación JWT, CORS y SignalR. También es donde se aplican las migraciones automáticamente al arrancar y donde se mapean los controllers y el hub del chat.

ChatHub: Esta es la clase que gestiona el chat en tiempo real mediante SignalR. Mantiene conexiones persistentes con los clientes mediante WebSockets.

1. Cuando un usuario entra en la página del chat, el cliente llama al método UnirseAVivienda que añade esa conexión a un grupo identificado por el ID de la vivienda, de forma que solo los usuarios de esa vivienda reciben los mensajes.
2. Cuando un usuario envía un mensaje, el servidor lo guarda en la base de datos y lo emite a todos los miembros del grupo mediante SendAsync, que invoca el método RecibirMensaje en todos los clientes conectados a esa vivienda.
3. En el cliente, Blazor escucha ese evento y actualiza la lista de mensajes en tiempo real sin necesidad de recargar la página.

CLIENT:

Program.cs (Client): Es el punto de entrada de la aplicación Blazor WebAssembly. Es donde se configura y registra todo lo necesario antes de arrancar la aplicación. En él se registran los servicios de inyección de dependencias como AuthService, AuthHandler y ILocalStorageService de Blazored.LocalStorage, se configura el HttpClient con la URL base de la API leída del appsettings.json y con el AuthHandler como handler para que todas las peticiones lleven el token JWT automáticamente, y se registran los servicios de MudBlazor.

wwwroot: Esta carpeta contiene todos los archivos estáticos de la aplicación: el index.html (en este archivo es posible cargar funciones JavaScript globales, en este caso se cargó una para bajar el chat al final de todo a la hora de enviar o recibir mensajes) que es el punto de entrada de Blazor WebAssembly y donde se cargan las fuentes, los CSS y el script de Blazor, la carpeta images con las imágenes de la aplicación, el app.css con los estilos globales y el appsettings.json con la URL de la API.

app.css: Contiene los estilos globales de la aplicación. Define las variables CSS con los colores, sombras, bordes y transiciones del sistema de diseño, los estilos de la página de login, las clases de utilidad, los estilos del chat, el calendario y las cards del menú.

MainLayout.razor: Este es el layout principal que envuelve todas las páginas de la aplicación excepto el login y el registro. Todas las páginas heredan este layout por defecto salvo las que usan @layout EmptyLayout, como las dos antes mencionadas.

NavMenu.razor: Es el componente del menú lateral de navegación. Contiene los enlaces a todas las páginas de la aplicación organizados en secciones, mostrando u ocultando opciones según el rol del usuario.

AuthService (Client): Se encarga de gestionar la sesión del usuario en el frontend. Al hacer login envía las credenciales a la API, recibe el token JWT y los datos del usuario y los guarda en dos claves separadas del localStorage: `jwt_token` con el token y `user_data` con el ID, nombre, email y rol del usuario. Esto permite obtener el token cuando se necesita para las peticiones HTTP y los datos del usuario cuando se necesita mostrar información en la interfaz, sin deserializar más datos de los necesarios.

AuthHandler: Se ejecuta automáticamente antes de cada petición HTTP. Su función es obtener el token JWT del localStorage y añadirlo como cabecera `Authorization: Bearer {token}` en cada petición, de forma que no hay que hacerlo manualmente en cada llamada a la API. Si el servidor devuelve un 401 (no autorizado), el handler limpia el localStorage y redirige al usuario al login.

AppTheme.cs: Se creó el archivo `AppTheme.cs` para centralizar la configuración del tema visual de MudBlazor en un único lugar. Define la paleta de colores primarios y secundarios, los colores de estado (éxito, error, advertencia, info), el color de fondo y el color de la AppBar.

Finalmente para la dockerización y despliegue de este proyecto web:

Se dockerizó la aplicación completa para garantizar que funciona de forma idéntica en cualquier entorno. Cualquier persona con Docker Desktop instalado puede levantar la aplicación ejecutando un único comando desde la raíz del proyecto.

La aplicación se divide en tres contenedores orquestados con docker-compose: SQL Server, la API y el cliente Blazor servido mediante nginx. Cada uno tiene su propio Dockerfile y el `docker-compose.yml` define el orden de arranque: SQL Server primero con un healthcheck, luego la API y finalmente el cliente. Los datos se persisten en un volumen Docker para que no se pierdan al parar los contenedores. Para el entorno Docker la API usa un `appsettings.Docker.json` que sobrescribe la cadena de conexión apuntando al contenedor de SQL Server.

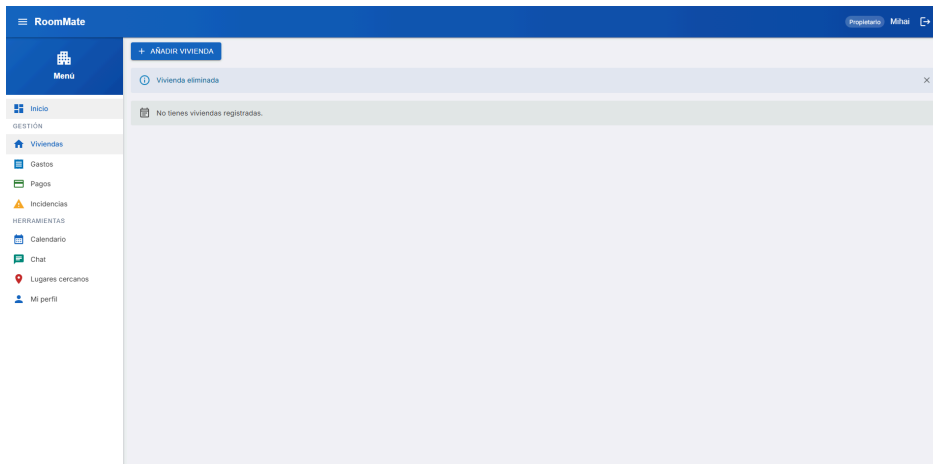
Para el despliegue en producción se usó un VPS de Hetzner con Ubuntu donde se instaló Docker, se clonó el repositorio de GitHub y se levantó la aplicación. Se configuró el subdominio gratuito `roommate-app.duckdns.org` en DuckDNS apuntando a la IP del servidor para que la aplicación fuese accesible desde internet.

Demostración:

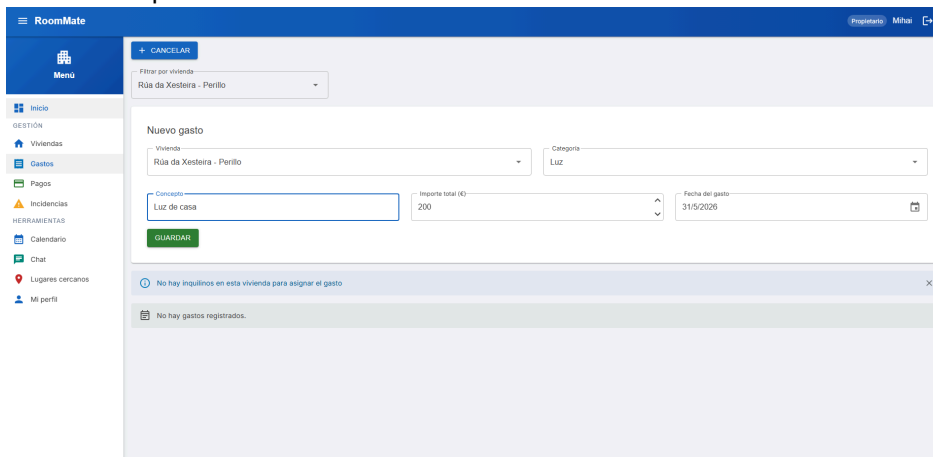
Añadir vivienda:

The screenshot displays the 'RoomMate' web application interface. On the left is a sidebar menu with options like 'Inicio', 'Gestión', 'Viviendas', 'Gastos', 'Pagos', 'Incidentes', and 'Herramientas'. The main content area is titled 'Añadir vivienda' and contains a form with the following fields: 'Calle' (filled with 'Rúa da Xesteira'), 'Número' (filled with '2'), 'Piso' (filled with '2'), 'Puerta' (filled with '10'), 'Ciudad' (filled with 'Perillo'), 'Código postal' (filled with '15172'), and 'Descripción'. A green 'GUARDAR' button is at the bottom of the form. Below the form, a blue notification banner states 'Vivienda creada'. Underneath this, a white card displays the details of the created housing: 'Rúa da Xesteira 2', '2 trq', and 'Perillo - 15172', with a red status indicator and the word 'INQUILINOS'.

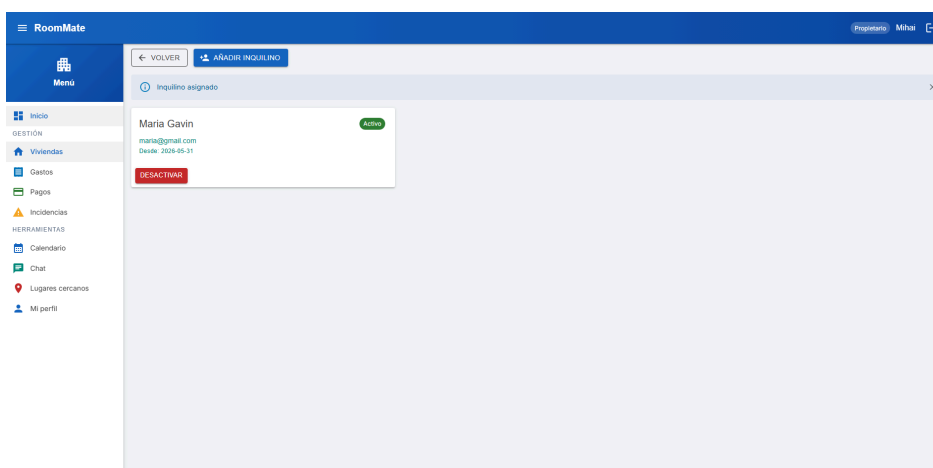
Eliminar vivienda:



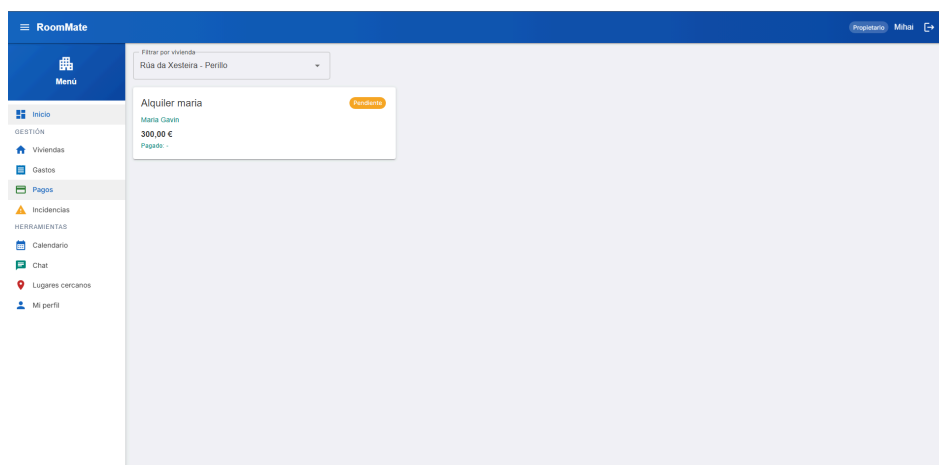
Gasto sin inquilinos:



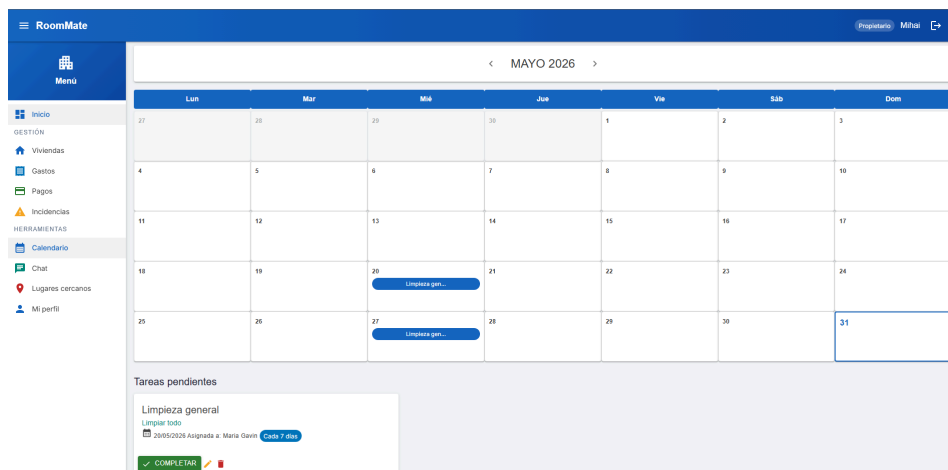
Añadir inquilino:



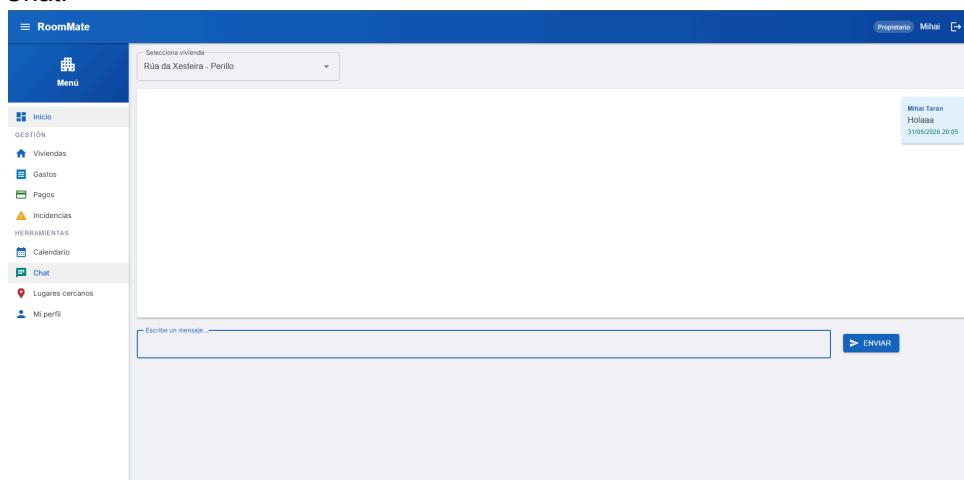
Pagos:



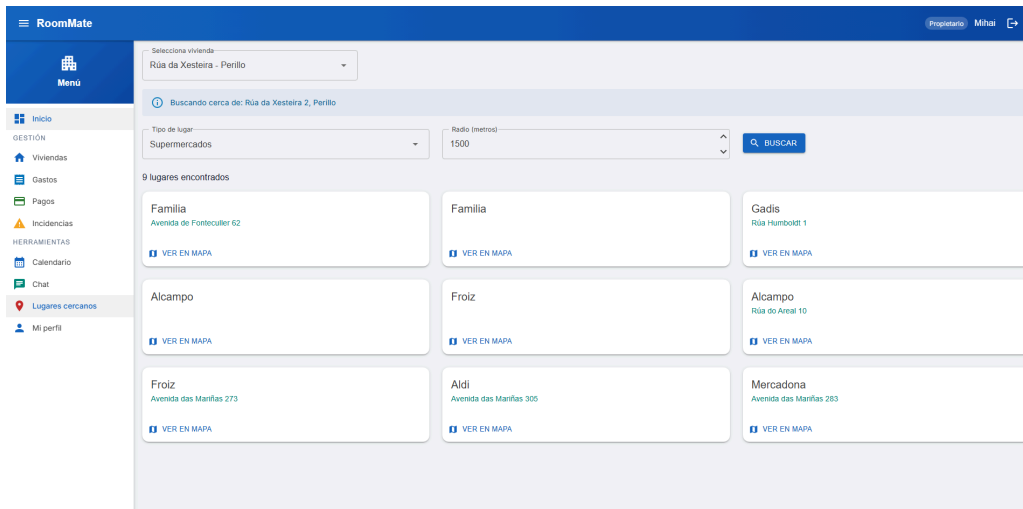
Tarea calendario:



Chat:



Lugares cercanos:



La web se encuentra desplegada para su prueba en <http://roommate-app.duckdns.org/login>.